CI/CD Deployment Assignment

You are required to deploy a **Flask backend** and an **Express frontend** on an **Amazon EC2 instance**. Additionally, implement a **CI/CD pipeline** using Jenkins to automate the deployment process.

Part 1: Deploy Flask and Express on a Single EC2 Instance

1. **Objective**:
    - Deploy both the Flask backend and the Express frontend on a **single Amazon EC2 instance**.
2. **Steps**:
    - **Provisioning the EC2 Instance**:
        - Launch an EC2 instance on AWS (you can use a free-tier eligible instance).
        - SSH into the instance and install the following dependencies:
            - Python for Flask.
            - Node.js for Express.
            - Git for pulling the application code.
    - **Application Setup**:
        - Clone the Flask and Express repositories onto the EC2 instance.
        - Install the required dependencies for both applications using `pip` and `npm`.
        - Configure both applications to run on different ports (e.g., Flask on port 5000 and Express on port 3000).
        - Start the applications using process managers like `pm2` or `systemd` to ensure they remain active.
3. **Deliverables**:
    - A running EC2 instance with Flask and Express accessible via the instance's public IP.
    - A description or diagram of the deployed architecture.

Project Structure:

```
flask-express-devops/
│
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── provider.tf
│
├── flask-backend/
│   ├── app.py
│   ├── requirements.txt
│   └── venv/              ← DID NOT PUSH
│
├── express-frontend/
│   ├── package.json
│   ├── package-lock.json
│   ├── server.js
│   └── node_modules/      ← DID NOT PUSH
│
├── Jenkinsfile
├── README.md
└── .gitignore
```

Final Architecture

User Browser

↓

Nginx :80

├── / → Express Frontend :3000

└── /api → Flask Backend :5000

Jenkins :8080

↓

Automated Deployment

STEP 1 — Create Project Folder on Your Linux Machine

Run on YOUR LOCAL LINUX MACHINE:

mkdir -p ~/flask-express-devops

cd ~/flask-express-devops

STEP 2 — Install Terraform

Run:

sudo apt update

sudo apt install -y gnupg software-properties-common curl unzip

curl -fsSL https://apt.releases.hashicorp.com/gpg | \

sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \

https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \

sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update

sudo apt install terraform -y

Verify:

terraform -version

```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops$ sudo apt install terraform -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be upgraded:
  terraform
1 upgraded, 0 newly installed, 0 to remove and 33 not upgraded.
Need to get 34.8 MB of archives.
After this operation, 156 kB of additional disk space will be used.
Get:1 https://apt.releases.hashicorp.com jammy/main amd64 terraform amd64 1.15.3-1 [34.8 MB]
Fetched 34.8 MB in 0s (211 MB/s)
(Reading database ... 235583 files and directories currently installed.)
Preparing to unpack .../terraform_1.15.3-1_amd64.deb ...
Unpacking terraform (1.15.3-1) over (1.15.2-1) ...
Setting up terraform (1.15.3-1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops$ terraform -version
Terraform v1.15.3
on linux_amd64
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops$ mkdir terraform
```

STEP 3 — Create Terraform Folder

mkdir terraform

cd terraform

```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops$ mkdir terraform
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops$ cd terraform/
```

STEP 4 — Create Terraform Files

---

provider.tf

```
terraform {
 required_providers {
   aws = {
     source = "hashicorp/aws"
     version = "~> 5.0"
   }
 }
}

provider "aws" {
 region = var.aws_region
}
```

---

variables.tf

```
variable "aws_region" {
 default = "ap-south-1"
}

variable "instance_type" {
 default = "t3.small"
```

```
}

variable "key_name" {
 default = "my-ec2-key"
}
```

---

main.tf

```
resource "aws_security_group" "devops_sg" {
 name = "flask-express-sg"

 ingress {
   description = "SSH"
   from_port   = 22
   to_port     = 22
   protocol    = "tcp"
   cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
   description = "HTTP"
   from_port   = 80
   to_port     = 80
   protocol    = "tcp"
   cidr_blocks = ["0.0.0.0/0"]
```

```
}

ingress {
  description = "Jenkins"
  from_port   = 8080
  to_port     = 8080
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Express"
  from_port   = 3000
  to_port     = 3000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Flask"
  from_port   = 5000
  to_port     = 5000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
```

```
 }

 egress {
   from_port   = 0
   to_port     = 0
   protocol    = "-1"
   cidr_blocks = ["0.0.0.0/0"]
 }
}

resource "aws_instance" "devops_server" {
 ami                    = "ami-0f58b397bc5c1f2e8"
 instance_type          = var.instance_type
 key_name               = var.key_name
 vpc_security_group_ids = [aws_security_group.devops_sg.id]

 tags = {
   Name = "Flask-Express-Server"
 }
}
```

---

outputs.tf

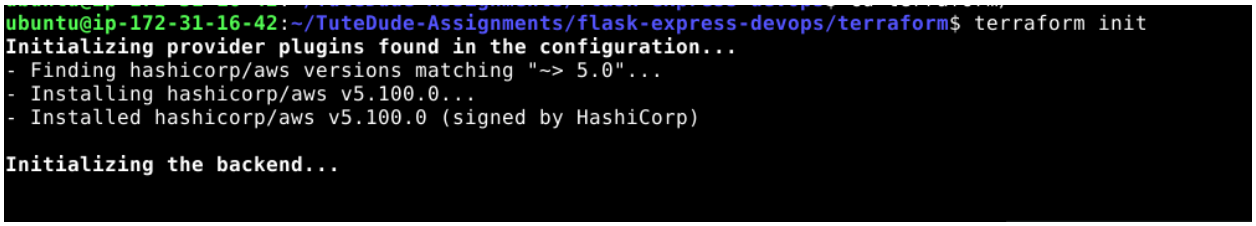
```
output "public_ip" {
 value = aws_instance.devops_server.public_ip
```

}

STEP 5 — Run Terraform

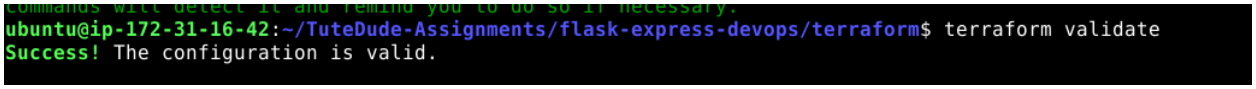
Initialize Terraform

terraform init



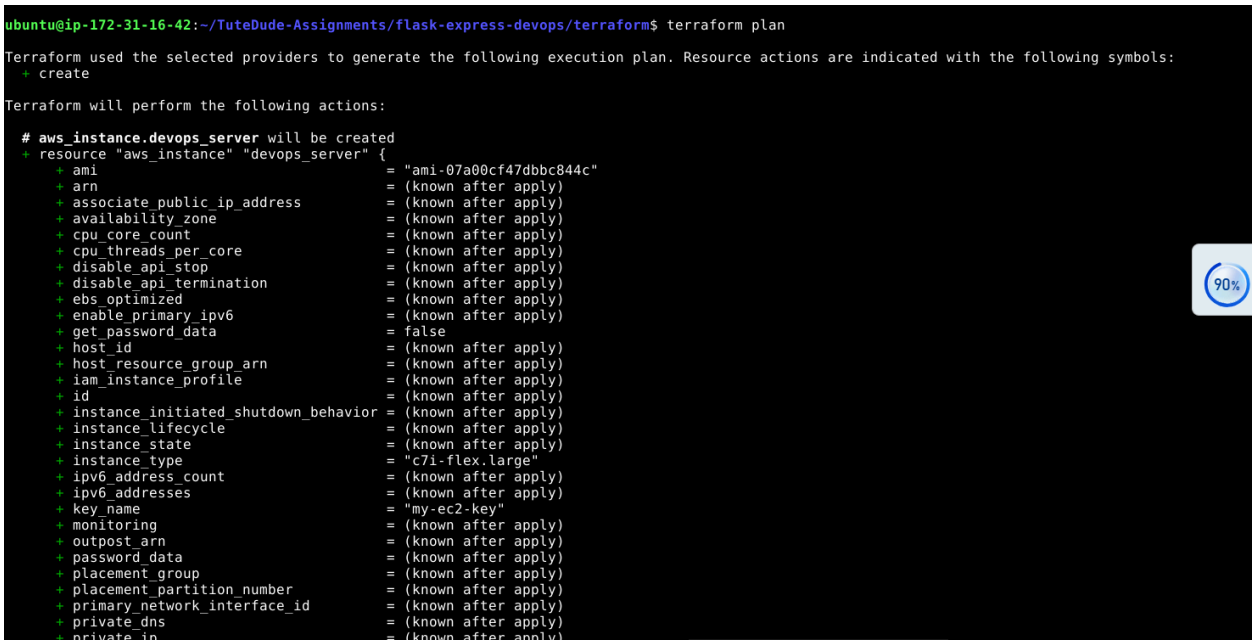
Validate Files

terraform validate



Preview Infrastructure

terraform plan

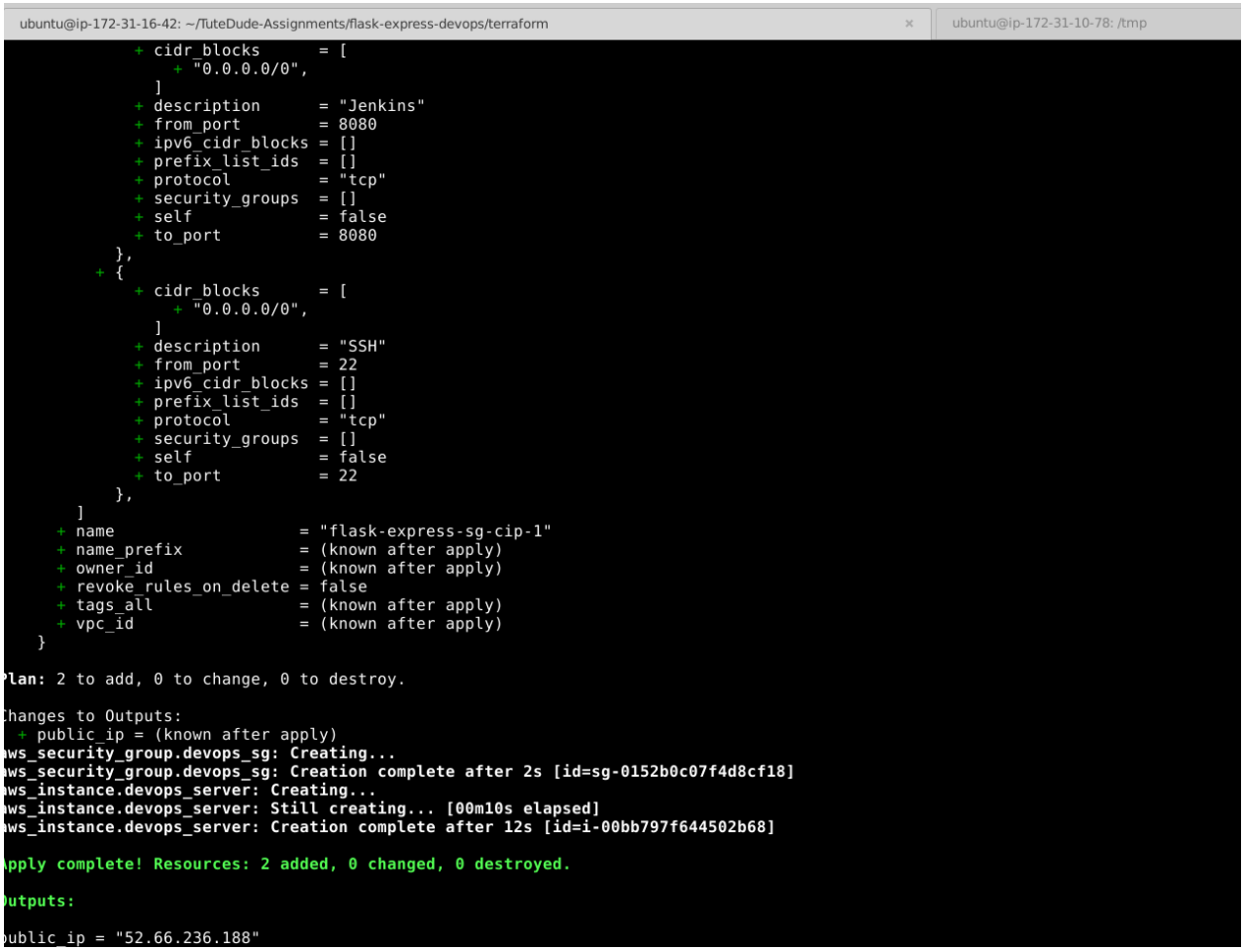


Create EC2 Server

terraform apply -auto-approve

public_ip = xx.xx.xx.xx

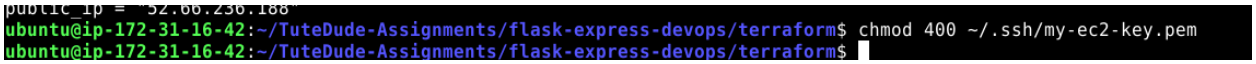
```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/flask-express-devops/terraform$ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.devops_server will be created
  + resource "aws_instance" "devops_server" {
      + ami                                  = "ami-07a00cf47dbbc844c"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "c7i-flex.large"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "my-ec2-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "Flask-Express-Server"
        }
      + tags_all                             = {
```


ubuntu@ip-172-31-16-42: ~/TuteDude-Assignments/flask-express-devops/terraform
ubuntu@ip-172-31-10-78: /tmp
+ cidr_blocks = [
+ "0.0.0.0/0",
]
+ description = "Jenkins"
+ from_port = 8080
+ ipv6_cidr_blocks = []
+ prefix_list_ids = []
+ protocol = "tcp"
+ security_groups = []
+ self = false
+ to_port = 8080
},
+ {
+ cidr_blocks = [
+ "0.0.0.0/0",
]
+ description = "SSH"
+ from_port = 22
+ ipv6_cidr_blocks = []
+ prefix_list_ids = []
+ protocol = "tcp"
+ security_groups = []
+ self = false
+ to_port = 22
},
]
+ name = "flask-express-sg-cip-1"
+ name_prefix = (known after apply)
+ owner_id = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all = (known after apply)
+ vpc_id = (known after apply)
}
Plan: 2 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ public_ip = (known after apply)
aws_security_group.devops_sg: Creating...
aws_security_group.devops_sg: Creation complete after 2s [id=sg-0152b0c07f4d8cf18]
aws_instance.devops_server: Creating...
aws_instance.devops_server: Still creating... [00m10s elapsed]
aws_instance.devops_server: Creation complete after 12s [id=i-00bb797f644502b68]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
Outputs:
public_ip = "52.66.236.188"

STEP 6 — Move Key File

Set permissions key file

chmod 400 ~/.ssh/my-ec2-key.pem



STEP 7 — SSH into EC2

ssh -i ~/.ssh/my-ec2-key.pem ubuntu@YOUR_PUBLIC_IP

STEP 8 — Update Ubuntu Server

sudo apt update -y

sudo apt upgrade -y

STEP 9 — Install Required Software

Install Python + Nginx + Git:

sudo apt install python3-pip python3-venv nginx git curl -y

STEP 10 — Install Node.js

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install nodejs -y

Verify:

node -v

npm -v

STEP 11 — Install PM2

sudo npm install -g pm2

Verify:

pm2 -v

STEP 12 — Create Flask Backend

cd ~

Create folder:

mkdir flask-backend

cd flask-backend

Create app.py

nano app.py

CODE:

```
from flask import Flask, jsonify


app = Flask(__name__)


@app.route('/api')

def home():

   return jsonify({
```

```
    "message": "Flask Backend Running Successfully!"
  })

if __name__ == '__main__':
  app.run(host='0.0.0.0', port=5000)
```

Create requirements.txt

nano requirements.txt

DATA:

Flask==3.0.3

gunicorn==22.0.0

Install Python Dependencies

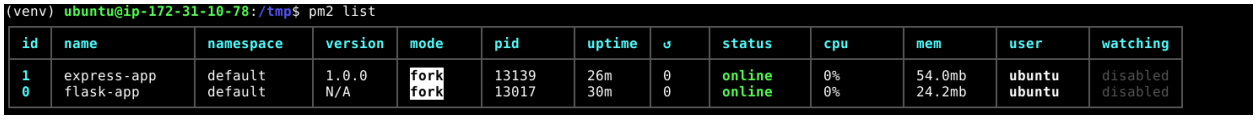
pip3 install -r requirements.txt

Start Flask App

pm2 start "gunicorn --bind 0.0.0.0:5000 app:app" --name flask-app

Check:

pm2 list



STEP 13 — Create Express Frontend

cd ~

mkdir express-frontend

cd express-frontend

Create package.json

nano package.json

CODE:

```
{
 "name": "express-frontend",
 "version": "1.0.0",
 "main": "server.js",
 "scripts": {
   "start": "node server.js"
 },
 "dependencies": {
   "express": "^4.19.2"
 }
}
```

Install Packages

npm install

Create server.js

nano server.js

code:

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.send(`
    <h1>Express Frontend Running</h1>
    <h2>DevOps Assignment Successful</h2>
    <p>Flask API available at /api</p>
  `);
});


app.listen(3000, '0.0.0.0', () => {
  console.log('Server running on port 3000');
});
```

Start Express App

pm2 start server.js --name express-app

STEP 14 — Save PM2 Services

pm2 save

Enable startup:

pm2 startup

STEP 15 — Configure Nginx Reverse Proxy

Edit config:

sudo nano /etc/nginx/sites-available/default

Code:

```
server {
  listen 80;
  location / {
     proxy_pass http://localhost:3000;

     proxy_http_version 1.1;

     proxy_set_header Upgrade $http_upgrade;
     proxy_set_header Connection 'upgrade';
     proxy_set_header Host $host;

     proxy_cache_bypass $http_upgrade;
  }

  location /api {
     proxy_pass http://localhost:5000;
  }
}
```

STEP 16 — Restart Nginx

sudo nginx -t

sudo systemctl restart nginx

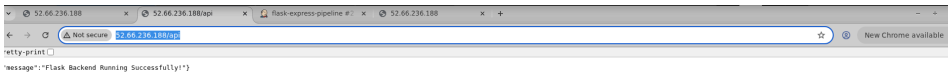
STEP 17 — Test Application

Frontend:

http://52.66.236.188/

Backend:

http://52.66.236.188/api



STEP 18 — Install Java for Jenkins

sudo apt install fontconfig openjdk-21-jre -y

Verify:

java -version

STEP 19 — Install Jenkins

Add key:

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \

sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

Add repo:

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \

https://pkg.jenkins.io/debian-stable binary/ | \

sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

Install:

sudo apt update

sudo apt install jenkins -y

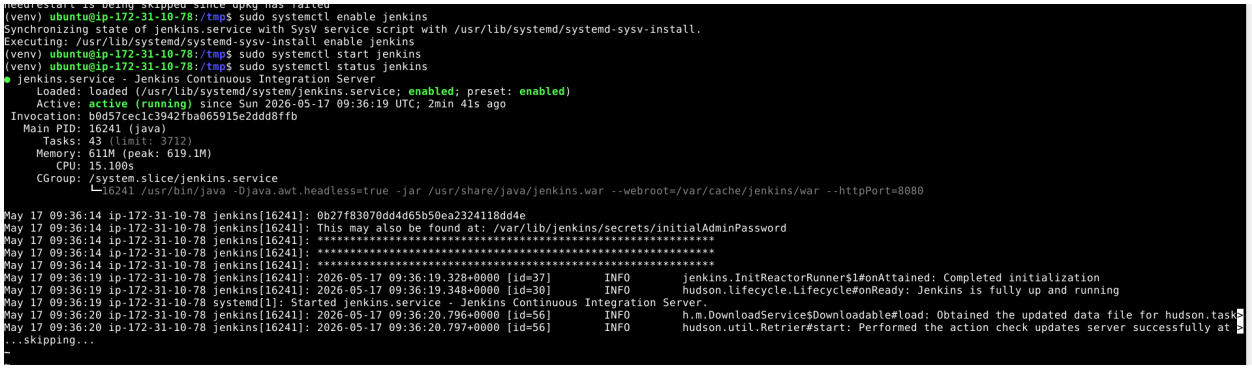
STEP 20 — Start Jenkins

sudo systemctl enable jenkins

sudo systemctl start jenkins

Check:

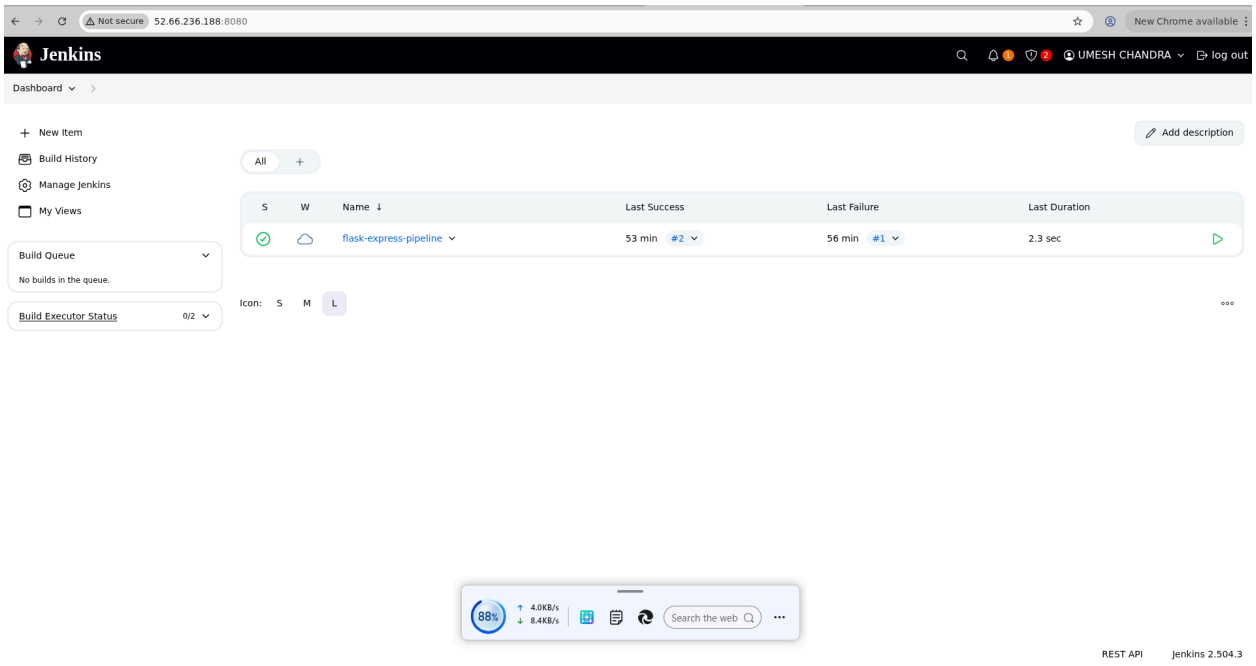
sudo systemctl status jenkins



STEP 21 — Open Jenkins

Open:

http://YOUR_PUBLIC_IP:8080



Get admin password:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Install:

- Suggested Plugins

Created admin account

STEP 22 — Give Jenkins Permissions

sudo usermod -aG sudo jenkins

sudo usermod -aG ubuntu jenkins

Edit sudoers:

sudo visudo

Add at bottom:

jenkins ALL=(ALL) NOPASSWD:ALL

Restart Jenkins:

sudo systemctl restart jenkins

```
(venv) ubuntu@ip-172-31-10-78:/tmp$ sudo usermod -aG sudo jenkins
(venv) ubuntu@ip-172-31-10-78:/tmp$ sudo usermod -aG ubuntu jenkins
(venv) ubuntu@ip-172-31-10-78:/tmp$ sudo visudo
(venv) ubuntu@ip-172-31-10-78:/tmp$ sudo systemctl restart jenkins
```

STEP 23 — Create Jenkins Pipeline Job

Inside Jenkins:

- **New Item**
- **Name:** `flask-express-pipeline`
- **Select: Pipeline**
- **Pipeline Script**

Code:

```
pipeline {

    agent any
```

```
    stages {
        stage('Restart Flask') {
            steps {
                sh '''
                sudo su - ubuntu -c "pm2 restart flask-app"
                '''
            }
        }
        stage('Restart Express') {
            steps {
                sh '''
                sudo su - ubuntu -c "pm2 restart express-app"
                '''
            }
        }
        stage('Restart Nginx') {
            steps {
                sh '''
                sudo systemctl restart nginx
                '''
            }
        }
    }
}
```

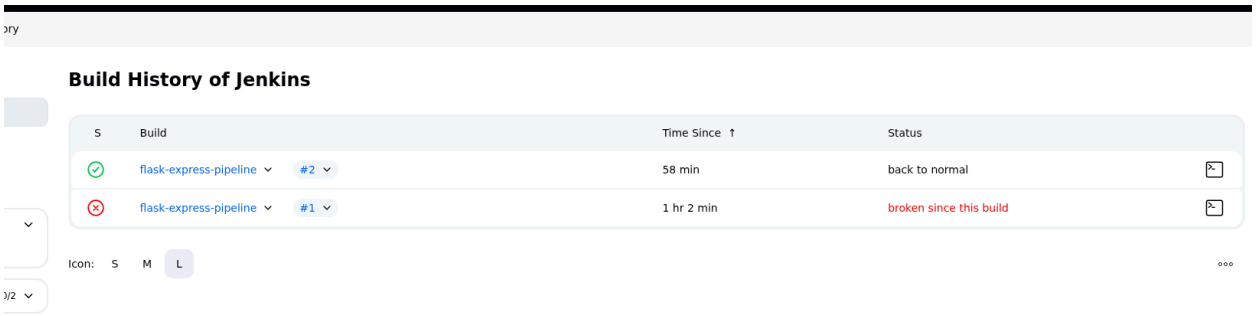
STEP 24 — Run CI/CD Pipeline

Click in jenkins portal:

Build Now

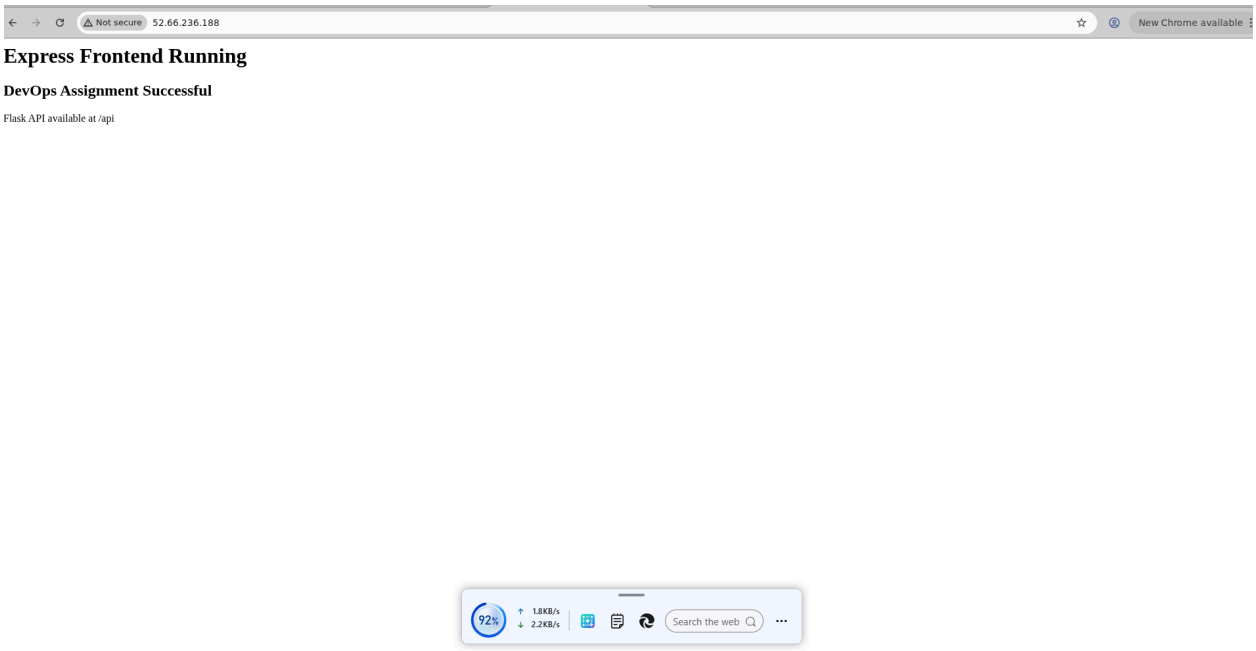
Check:

Console Output

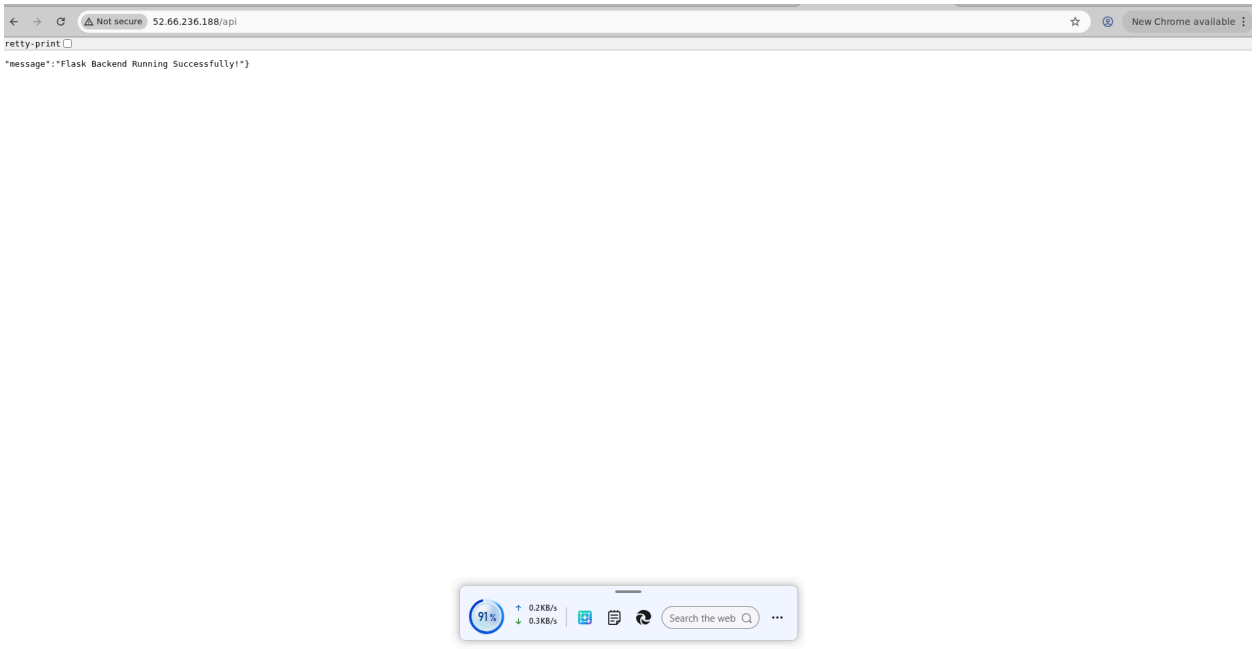


STEP 26 — Final Deliverables

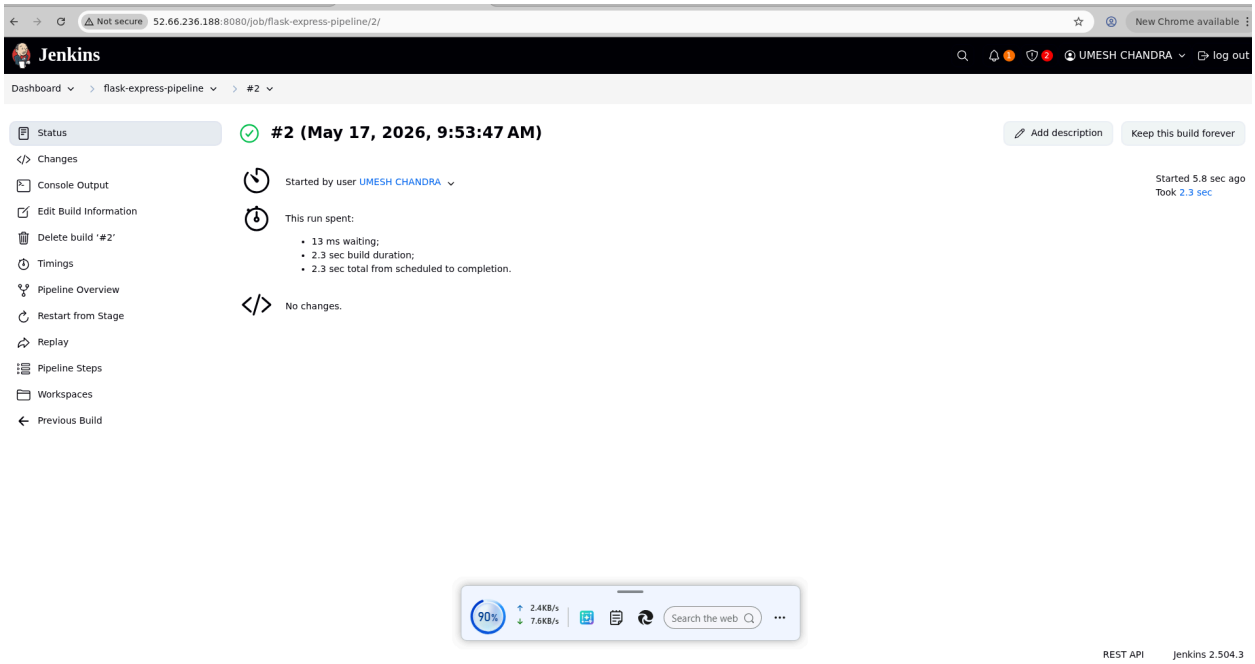
Frontend



Backend



Jenkins


Not secure 52.66.236.188:8080/job/flask-express-pipeline/2/
New Chrome available
Jenkins
UMESH CHANDRA
log out
Dashboard
flask-express-pipeline
#2
Status
Changes
Console Output
Edit Build Information
Delete build '#2'
Timings
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#2 (May 17, 2026, 9:53:47 AM)
Add description
Keep this build forever
Started by user UMESH CHANDRA
Started 5.8 sec ago
Took 2.3 sec
This run spent:
13 ms waiting;
2.3 sec build duration;
2.3 sec total from scheduled to completion.
No changes.
90%
2.4KB/s
7.6KB/s
Search the web
REST API
Jenkins 2.504.3

Part 2: Implement CI/CD Pipeline Using Jenkins

1. **Objective**:
    - Automate the deployment of Flask and Express applications using Jenkins.
2. **Steps**:
    - **Install Jenkins**:
        - Install Jenkins on the same EC2 instance or on a separate machine.
        - Configure Jenkins by installing essential plugins like Git, NodeJS, and Python.
    - **Set Up Jenkins Pipeline**:
        - Create two separate Jenkins pipelines for the Flask and Express applications.
        - **Pipeline Steps**:
            - Pull the latest code from the respective Git repositories.
            - Install dependencies for Flask (`pip install -r requirements.txt`) and Express (`npm install`).
            - Restart the applications using the process manager (e.g., `pm2 restart <app>`).
    - **Triggering the Pipeline**:
        - Set up a GitHub webhook to trigger the Jenkins pipeline on every push to the repositories.
        -
    - **Optional Enhancements**:
        - Add testing stages to the pipeline for both applications.
        - Configure environment variables in Jenkins for managing sensitive data like API keys.
3. **Deliverables**:
    - A fully functional CI/CD pipeline that automates the deployment process for Flask and Express.
    - A Jenkins pipeline script (`Jenkinsfile`) for each application.
    - Evidence of the pipeline working (e.g., screenshots of successful builds and deployments).

GIT HUB URL :

Project Structure

```
devops-assignment2/
│
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── provider.tf
│
├── flask-app/
│   ├── app.py
│   ├── requirements.txt
│   ├── Jenkinsfile
│   └── README.md
│
├── express-app/
│   ├── server.js
│   ├── package.json
│   ├── Jenkinsfile
│   └── README.md
│
└── screenshots/
```

STEP 1 — Create Terraform Files

---

terraform/provider.tf

```
provider "aws" {
 region = "ap-south-1"
}
```

---

terraform/variables.tf

```
variable "instance_type" {
 default = "t3.small"
}

variable "key_name" {
 default = "my-ec2-key"
}
```

---

terraform/main.tf

```
resource "aws_security_group" "devops_sg" {
 name = "devops-security-group"

 ingress {
   from_port   = 22
```

```
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  from_port   = 5000
  to_port     = 5000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  from_port   = 3000
  to_port     = 3000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  from_port   = 8080
  to_port     = 8080
  protocol    = "tcp"
```

```
  cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }
}

resource "aws_instance" "devops_server" {
 ami                    = "ami-0f58b397bc5c1f2e8"
 instance_type          = var.instance_type
 key_name               = var.key_name
 vpc_security_group_ids = [aws_security_group.devops_sg.id]
```

```
 tags = {
   Name = "DevOps-Assignment-Server"
 }
}
```

---

terraform/outputs.tf

```
output "public_ip" {
 value = aws_instance.devops_server.public_ip
}
```

STEP 2 — Create EC2 using Terraform

Open terminal:

```
cd terraform
terraform init
terraform plan
terraform apply
```

Type:

yes

STEP 3 — Connect to EC2

chmod 400 my-ec2-key.pem

ssh -i my-ec2-key.pem ubuntu@YOUR_PUBLIC_IP

STEP 4 — Install Required Software on EC2

---

sudo apt update

sudo apt upgrade -y

Install Git

sudo apt install git -y

Install Python

sudo apt install python3-pip python3-venv -y

Install NodeJS

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install nodejs -y

Checking version:

node -v

npm -v

Install PM2

sudo npm install -g pm2

Install Jenkins

sudo apt install openjdk-17-jdk -y

Install Jenkins

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \

/usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \

https://pkg.jenkins.io/debian-stable binary/ | sudo tee \

/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins -y

Start Jenkins

sudo systemctl enable jenkins

sudo systemctl start jenkins

sudo systemctl status jenkins

STEP 5 — Open Jenkins

http://3.110.152.242:8080

Get admin password:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Paste password.

Install suggested plugins.

Creating an admin user.

Install Jenkins Plugins

Manage Jenkins

→ Plugins

Install:

- Git
- Pipeline
- NodeJS
- SSH Agent

Restart Jenkins.

STEP 6— Create Flask Application

---

flask-app/app.py

```
from flask import Flask


app = Flask(__name__)


@app.route("/")
def home():
  return {
    "message": "Flask Backend Running Successfully"
  }


if __name__ == "__main__":
  app.run(host="0.0.0.0", port=5000)
```

---

flask-app/requirements.txt

```
flask
gunicorn
```

---

flask-app/Jenkinsfile

```
pipeline {
  agent any

  stages {

    stage('Clone Repository') {
      steps {
        git 'YOUR_FLASK_GITHUB_REPO_URL'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh '''
        python3 -m venv venv
        . venv/bin/activate
        pip install -r requirements.txt
        '''
      }
    }

    stage('Deploy Flask App') {
```

```
        steps {
            sh '''
            pm2 delete flask-app || true

            pm2 start "venv/bin/python app.py" --name flask-app

            pm2 save
            '''
        }
    }
  }
}
```

STEP 7 — Create Express Application

---

express-app/server.js

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.json({
```

```
      message: "Express Frontend Running Successfully"
    });
});


app.listen(3000, '0.0.0.0', () => {
    console.log('Server running on port 3000');
});
```

---

express-app/package.json

```
{
  "name": "express-app",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

---

express-app/Jenkinsfile

```
pipeline {
  agent any

  stages {

    stage('Clone Repository') {
      steps {
        git 'YOUR_EXPRESS_GITHUB_REPO_URL'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh '''
        npm install
        '''
      }
    }

    stage('Deploy Express App') {
```

```
        steps {
            sh '''
            pm2 delete express-app || true

            pm2 start server.js --name express-app

            pm2 save
            '''
        }
    }
  }
}
```

STEP 8 — Push Flask App

Inside `flask-app`:

```
git init
git add .
git commit -m "Initial Flask commit"
git branch -M main
git remote add origin https://github.com/UmeshChandra2001/devops-assignment2.git
git push -u origin main
```

Push Express App

Inside `express-app`:

git init

git add .

git commit -m "Initial Express commit"

git branch -M main

git remote add origin https://github.com/UmeshChandra2001/devops-assignment2.git

git push -u origin main

STEP 9 — Configure Jenkins Pipelines

Flask Pipeline

Jenkins:

New Item

→ **Pipeline**

→ **Name: flask-pipeline**

Pipeline:

Pipeline script from SCM

SCM:

Git

Repository URL:

https://github.com/UmeshChandra2001/devops-assignment2.git

Script Path:

flask-app/Jenkinsfile

Express Pipeline

New Item

→ **Pipeline**

→ **Name: express-pipeline**

Pipeline:

Pipeline script from SCM

SCM:

Git

Repository URL:

https://github.com/UmeshChandra2001/devops-assignment2.git

Script Path:

express-app/Jenkinsfile

STEP 10 — Fix Jenkins Permission Issue

SSH into EC2:

sudo usermod -aG sudo jenkins

sudo chown -R jenkins:jenkins /var/lib/jenkins

sudo chmod -R 755 /var/lib/jenkins

Restart Jenkins:

sudo systemctl restart jenkins

STEP 11 — Give PM2 Access to Jenkins

sudo su - jenkins

pm2 startup

exit

sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u jenkins --hp /var/lib/jenkins

---

STEP 12 — Configure GitHub Webhook

GitHub Repository:

Settings

→ Webhooks

→ Add webhook

Payload URL:

http://3.110.152.242:8080/github-webhook/

Content type:

application/json

Event:

Just the push event

STEP 13— Enable GitHub Hook Trigger in Jenkins

Pipeline:

Configure

→ Build Triggers

→ GitHub hook trigger for GITScm polling

STEP 14— Run Pipelines

Push code:

git add .

git commit -m "deploy update"

git push

Jenkins automatically deploys.

Verify Applications

Flask

http://3.110.152.242:5000/

Express Frontend Running

DevOps Assignment Successful

Flask API available at /api

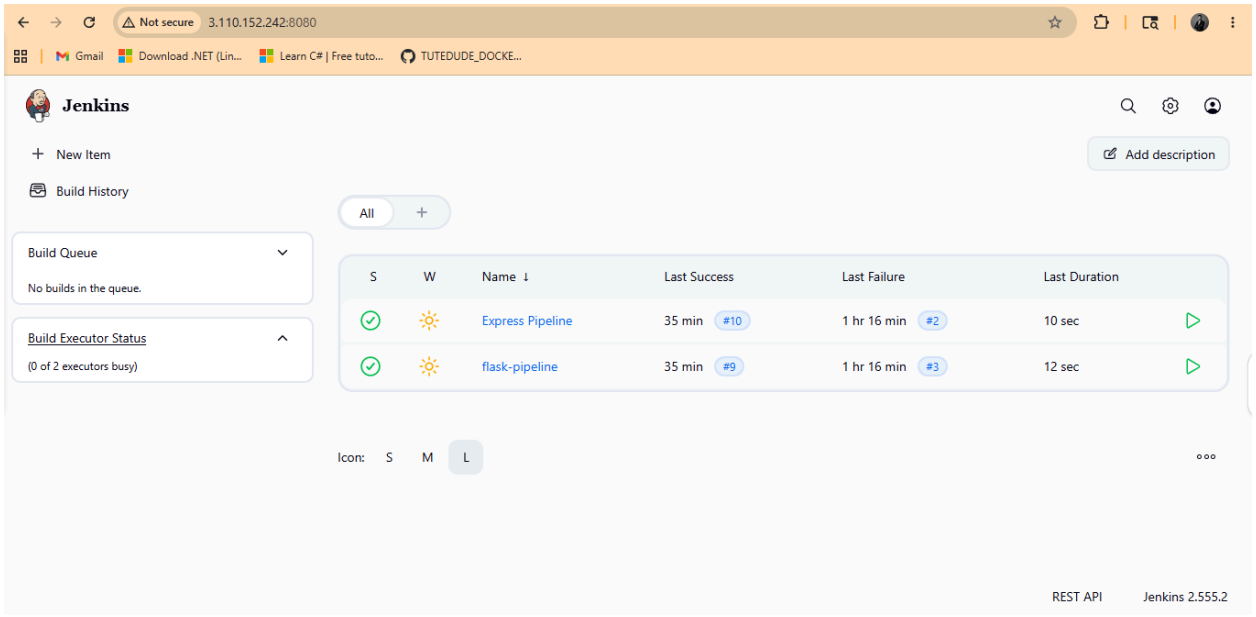
Express

http://3.110.152.242:3000/

retty-print ☐

"message":"Flask Backend Running Successfully!"}

JENKINS


Not secure 3.110.152.242:8080
Gmail
Download .NET (Lin...
Learn C# | Free tuto...
TUTEDUDE_DOCKE...
Jenkins
New Item
Build History
Add description
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
All
S
W
Name ↓
Last Success
Last Failure
Last Duration
Express Pipeline
35 min #10
1 hr 16 min #2
10 sec
flask-pipeline
35 min #9
1 hr 16 min #3
12 sec
Icon: S M L
REST API
Jenkins 2.555.2

Submission Requirements

1. A link to the GitHub repositories for Flask and Express.
2. A document (`README.md`) describing the deployment process and CI/CD pipeline setup.
3. Screenshots of:
    - The running EC2 instance with Flask and Express accessible.
    - Jenkins pipeline execution logs showing successful deployment.